

Statistical Programming Languages

Introduction

Katarzyna Reluga¹

¹Acknowledgements: Thanks to Manuel Pfeuffer, Maarten Jung, and Tobias Wistub for sharing notes from earlier editions.

What is R?

- ▶ Programming environment mainly used for data manipulation, calculation and graphical display
- ▶ R can be regarded as an implementation of the S language (developed by Becker and Chamber in 1984)
- ▶ Written by R. Ihaka and R. Gentleman in New Zealand in 1990-1992
- ▶ Open source
- ▶ Great variety of packages covering all fields of statistics and machine learning (and more)
- ▶ Versions for Windows, Linux/Unix, Mac OS

Installing and using R

- ▶ R available from the [Comprehensive R Archive Network \(CRAN\)](#)
- ▶ Do not use the R graphical user interface (GUI)² application for development
- ▶ Use a good text editor or an integrated development environment (IDE)³ instead
 - ▶ [RStudio](#) (Highly recommended and mandatory in this course!)
 - ▶ VS Code with [R extension](#)
 - ▶ Emacs with [ESS](#)
 - ▶ Sublime with [Sublime Text 2](#)

²GUI is the visual layer of a program: windows, buttons, menus, icons — all the things you can click with a mouse.

³IDE is a software tool used by programmers to write, test, and debug code. Often has a GUI (menus, toolbars, editor windows), but “IDE” refers to the whole integrated system, not just the visual interface.

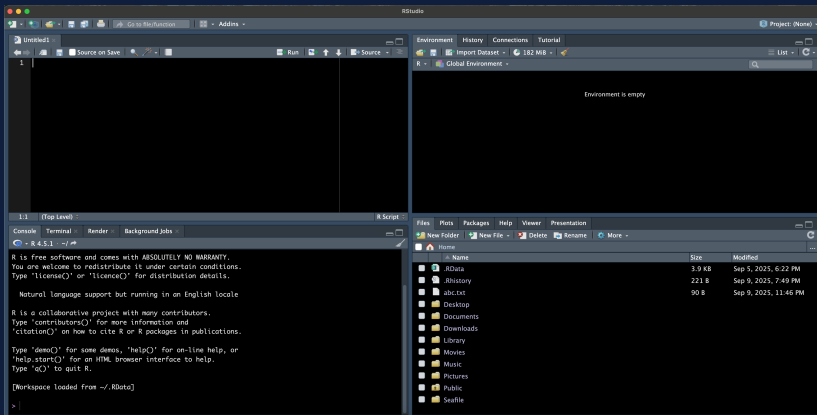


Figure 1: Standard RStudio console with standard editor running on Windows.

Binary mathematical functions

```
1 + 2      # addition
## [1] 3

1 - 2      # subtraction
## [1] -1

1 * 2      # multiplication
## [1] 2

1 / 2      # division
## [1] 0.5

1 ^ 2      # power
## [1] 1

5 %/% 2    # integer division
## [1] 2

5 %% 2     # modulo
## [1] 1
```

Mathematical functions (1)

```
sqrt(2)
## [1] 1.414214
sin(pi)      # cos, tan
## [1] 1.224647e-16
acos(0)      # asin, atan, atan2
## [1] 1.570796
exp(0)       # exponent
## [1] 1
log(1)       # natural logarithm
## [1] 0
```

Mathematical functions (2)

```
pi
## [1] 3.141593
floor(pi)      # largest integer <= pi --> 3
## [1] 3
ceiling(pi)    # smallest integer >= pi --> 4
## [1] 4
trunc(pi)      # integer part closer to 0 --> 3
## [1] 3
trunc(-pi)     # integer part closer to 0 --> -3
## [1] -3
floor(-pi)     # largest integer <= - pi --> -4
## [1] -4
round(pi)      # nearest integer --> 3
## [1] 3
```

Logical operators

- ▶ < - smaller
- ▶ <= - smaller or equal
- ▶ > - larger
- ▶ >= - larger or equal
- ▶ != - unequal
- ▶ == - equal

```
2 < 3  
## [1] TRUE
```

- ▶ & - logical AND (vector)⁴
- ▶ | - logical OR (vector)
- ▶ && - logical AND (no vector)
- ▶ || - logical OR (no vector)

```
2 < 3 & 2 > 4  
## [1] FALSE
```

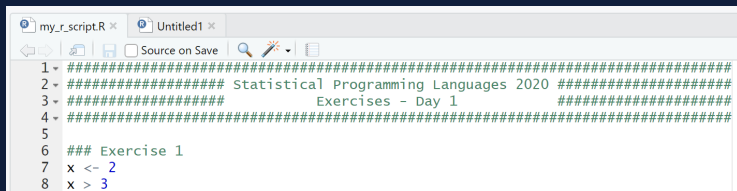
⁴In R, the operators & (AND) and | (OR) work on logical values (TRUE/FALSE). When they are applied to numeric vectors, R automatically coerces numbers to logicals: 0 → FALSE, any non-zero number → TRUE.

Writing R scripts

- ▶ Code can be stored in files with ending .R
- ▶ To run single lines from a script press Ctrl and Enter
- ▶ Use # to write comments

```
# This line is a comment  
2 + 3 # the calculation not  
## [1] 5
```

- ▶ Comments are not only helpful to explain your code, but also to structure it



The screenshot shows an R script editor with two tabs: 'my_r_script.R' and 'Untitled1'. The 'my_r_script.R' tab is active, displaying a script with the following content:

```
1 #####  
2 ##### Statistical Programming Languages 2020 #####  
3 ##### Exercises - Day 1 #####  
4 #####  
5  
6 ### Exercise 1  
7 x <- 2  
8 x > 3
```

Figure 2: Using comments to structure a R script.

Objects

- ▶ An object in R can be (nearly) everything, for example, a scalar, a matrix, a function, or even a plot
- ▶ The name of an object can be every combination of letters, numbers, and special characters. Names should begin with a letter
- ▶ Names **are** case sensitive (!)
- ▶ Overwriting objects causes **no** warning (!)

Variable assignments

```
a <- 2  # name <- assigned_value
b = 2   # name = assigned_value
2 -> c  # assigned_value -> name
d = e = f = 3
assign("y", 70 / 4)
c; e; y # not recommended
## [1] 2
## [1] 3
## [1] 17.5
```

`<-` is recommended in Hadley Wickham's [style guide](#).

My personal usage:

- ▶ `<-` for functions
- ▶ `=` for variables, vectors, matrices, etc

Overview of objects

```
ls()          # lists objects; identical: objects()
## [1] "a" "b" "c" "d" "e" "f" "y"

rm(a)         # removes a from workspace
ls()
## [1] "b" "c" "d" "e" "f" "y"
rm(list = ls()) # removes all objects
ls()
## character(0)
```

R basic data types

Use `mode(<object>)` to get the object's type.

- ▶ *character* - a string of arbitrary characters (always enclosed in quotation marks ' ' or " ")
- ▶ *numeric* - real number
- ▶ *logical* - TRUE or FALSE
- ▶ *complex* - complex number
- ▶ *function*
- ▶ *list*

```
mode("Hallo"); mode(pi); mode(TRUE); mode(1 + 3i)
## [1] "character"
## [1] "numeric"
## [1] "logical"
## [1] "complex"
```

R main structures

- ▶ *vector* - one-dimensional array of length m , all elements must be of the same type
- ▶ *matrix* - two-dimensional $m \times n$ arrays; array, all elements of the same type
- ▶ *array* - generalisation of matrices to more than two dimensions
- ▶ *data frame* - two-dimensional structure, like a matrix but allows different types in different columns
- ▶ *list* - collection of objects of arbitrary types and structures

R classes

- ▶ The *class* of an object determines how it is treated (object-oriented programming)
- ▶ Use `class(<object>)` to get the object's class
- ▶ All object types correspond to classes
- ▶ *vector* is not its own class;
`class(<vector>)` returns the type of its elements (e.g., "numeric", "character")
- ▶ All other main structures listed above are distinct classes
- ▶ Other important classes include:
 - ▶ *integer* - integer values
 - ▶ *factor* - categorical data stored as integers with associated labels

is.*() and as.*()

- ▶ There are a lot of functions of the form `is.*()` and `as.*()`
- ▶ They are available for classes and more (in particular, `is.vector()` and `as.vector()` exist)
- ▶ `is.*(<object>)` checks, whether an object is of kind *
- ▶ `as.*(<object>)` attempts to coerce an object to kind *

```
is.numeric(1)
## [1] TRUE
is.character(1)
## [1] FALSE
as.character(1)
## [1] "1"
```


Important R functions

Workspace: all objects (vectors, scalars, functions, etc.) created in a session (RStudio: **Environment** tab)

History: all commands typed in a session (RStudio: **History** tab)

```
setwd("x:/myR") # set working directory
getwd()         # get working directory
save.image()    # saves workspace to .Rdata
savehistory()   # save command history to .Rhistory
load(".Rdata")  # loads workspace
loadhistory()   # enables to browse the history in
                # command line using up and down
source("abc.R") # executes code from a file
sink("abc.txt") # creates file abc.txt and captures
3 + 5           # the console output into a file
sink()          # closes the sink --> output back
q(); quit()     # quits R
```

R packages

- ▶ Packages contain functions and datasets.
- ▶ Before usage, you need to install a package with `install.packages("<package>")` (only once).
- ▶ To access a package's content, you must load and attach it with `library("<package>")` or `require("<package>")` (each time you start a new session).
- ▶ `library()` throws an *error* if the package is not found, while `require()` only gives a *warning* and returns `FALSE` (preferable inside functions).
- ▶ If you only want to use a single function from a package, you don't need to load it. Instead, call it with `<package>::<function>`.

Installing new packages

```
install.packages("mvtnorm") # download and install
update.packages()           # update local packages
remove.packages("mvtnorm")  # remove installed package

library()                   # show installed libraries
library("mvtnorm")          # load and attach library
require("mvtnorm")          # load and attach library

# Use dmnorm without loading/attaching mvtnorm before
mvtnorm::dmnorm(x = c(0, 0))

search()                    # show attached packages and data frames
ls(2)                       # show functions for second package
```

Getting Help

- ▶ `help()` displays the help page for a topic (e.g., `help(sin)`)
- ▶ `?` is equivalent to `help()` (e.g., `?sin`)
- ▶ `help.search()` searches the help system for a character string (e.g., `help.search("sin")`)
- ▶ `??` is equivalent to `help.search()` (e.g., `??sin`)
- ▶ `help.start()` opens the HTML version of R's documentation
- ▶ `library(help = "<package>")` displays information about a package (e.g., `library(help = "mvtnorm")`)

More Help

- ▶ Google (or your other favorite search engine)
- ▶ [Stack Overflow](#)
- ▶ [Quick-R](#)
- ▶ [R mailing lists](#): R-help, R-announce, R-packages, R-devel (for developer-specific topics), R-sig-* special interest groups (r-sig-finance)

For Further Reading

- ▶ An Introduction to R: [R-intro.pdf](#)
- ▶ The R Reference Index: [fullrefman.pdf](#)
- ▶ [Hadley Wickham's styleguide](#)
- ▶ Introductory Statistics with R, Peter Dalgaard
- ▶ R by Example, Jim Albert and Maria Rizzo
- ▶ Data Analysis and Graphics Using R, Maindonald/Braun
- ▶ R Graphics, Murrell
- ▶ Efficient R Programming, Colin Gillespie and Robin Lovelace
- ▶ Programmieren mit R, Uwe Ligges